QCViz Lab Meeting Presentation — Full English Script

QCViz Lab Meeting Presentation — Full English Script

[Slide 1: Why Experimentalists Need Computation — and Why They Can't Easily Get It]

Every day, experimentalists ask questions that need electronic structure information.

"Where is the HOMO in this molecule?" [pause] "If I change this substituent, how do the HOMO and LUMO shift?" [pause] "Which side is electron-rich on the ESP map?" [pause] "What happens to this geometry after optimization?"

These are not big computational projects. These are quick, concrete questions that come up in the middle of experiments.

But to answer these with computation, the question must pass through seven separate steps. [pause]

Structure preparation. Input file writing. Setting up calculation parameters. Running the engine. Parsing the output. Opening a separate viewer for visualization. And finally, interpretation.

The bottleneck is not the speed of the compute engine. [pause] The manual work before and after the engine takes most of the total time.

And if the question changes even slightly, you repeat most of these steps from scratch.

Takeaway: The problem is not computation itself. The problem is the seven-step path between a question and an answer.

[Slide 2: The Real Cost — A Vicious Cycle in the Lab]

This fragmented path creates three real costs in the lab.

First, slow turnaround. [pause] The time from question to first result is so long that people simply use computation less. Not because computation is less valuable. Because the friction of using it once is too high.

Second, workload concentration. [pause] If only a few people can set up calculations, every request goes to them. Those people spend their time writing repetitive input files and post-processing results. They lose time for interpretation and experiment design — the work that actually needs their expertise.

Third, computation stays outsourced. [pause] Experimentalists never get the chance to build computational intuition. Computation becomes one person's job, not the lab's shared tool.

This is not a performance problem of the engine. This is an access-path problem from question to result.

Takeaway: The cost is not compute time. The cost is human time lost to friction.

[Slide 3: QCViz — One Browser, One Conversation, Real Computation]

QCViz solves this entire path in one browser window, with no interruption.

The user asks a question in natural language. [pause]

The system identifies the molecule and gets a 3D structure automatically. It selects calculation settings based on the question type and molecular properties. PySCF (pie-S-C-F) runs a real DFT or Hartree–Fock calculation. Results come back inside the browser — 3D visualization, energy values, convergence data, and interpretation metrics. [pause] If the user asks a follow-up question, the structure is reused and the session continues.

Two things to be clear about. [pause]

This is not an AI chatbot that makes up numbers. Every number comes from PySCF.

And this is not one convenience feature added on top. This is a redesign of the access path — from seven steps down to one.

Takeaway: QCViz replaces the seven-step path with a single conversation in a browser.

[Slide 4: This Type of Platform Already Publishes in High-Impact Journals]

A common question: "Can a web platform be a paper?" [pause] The answer is — it already is.

Let me show some examples.

SWISS-MODEL published repeatedly in Nucleic Acids Research from 2003 to 2018. It has tens of thousands of citations. It democratized protein structure modeling on the web.

Galaxy published in Nucleic Acids Research from 2005 to 2022. It automates bioinformatics workflows as a web platform.

WebMO appeared in WIREs Computational Molecular Science in 2021 — a web-based quantum chemistry interface for education and research.

AQME (AK-mee) appeared in the same journal in 2023 — Python-based workflow automation for computational chemistry.

ChemCrow published in Nature Machine Intelligence in 2024 — an LLM agent integrated with chemistry tools.

El Agente Q published in Matter from Cell Press in 2025 — a multi-agent LLM system for automated quantum chemistry.

ChemGraph published in Communications Chemistry from Nature in 2026 — an LLM agentic framework for DFT and molecular dynamics.

[pause]

Now, here is the key point. None of these systems combine all of the following in one seamless experience: a web platform, a natural language interface, automatic structure retrieval, automatic setting selection, real DFT execution, and browser-native visualization. [pause] That gap is the research space QCViz fills.

El Agente and ChemGraph are agents — CLI-based, designed for experts. QCViz is a platform — browser-based, designed for experimentalists.

What SWISS-MODEL did for protein modeling, QCViz aims to do for electronic structure calculation.

Takeaway: Web platforms that lower barriers have a strong publication track record. QCViz fills a specific gap none of them cover.

[Slide 5: QCViz at a Glance — The Full Pipeline in One Slide]

This slide shows the full architecture in one view.

Here is the pipeline flow. [pause]

The user asks a question in natural language. LangChain (LANG-chain) handles orchestration — it parses the intent and routes the task. MolChat (MOL-chat) grounds the molecule and gets the 3D structure. The automatic setup module selects the functional, basis set, and grid. PySCF runs the actual DFT or Hartree–Fock calculation. The results are packaged and rendered in the browser — visualization, energy data, and interpretation metrics.

To the user, this looks like one conversation. [pause] But inside, each role is separated.

From this slide onward, I will open each block one by one. Use this diagram as a map to follow where each block sits in the whole system.

Takeaway: One conversation on the outside. Separated, specialized modules on the inside.

[Slide 6: MolChat — From Any Name to a Compute-Ready 3D Structure, Automatically]

Let me start with the first block: MolChat. [pause]

Here is the existing bottleneck. To run any calculation, you need a compute-ready 3D coordinate. Getting that coordinate is a major bottleneck.

The 3D structure databases themselves are highly fragmented — PubChem (PUB-kem), ChemSpider, CSD, PDB, and various local databases. Even computational chemists spend time asking, "Where do I get the 3D coordinates for this molecule?"

For experimentalists, it is harder. They must write SMILES (smylz) strings manually, draw structures in an editor, or find and download files.

And if the input is not an exact name — say, "that ketone," a Korean abbreviation, or a descriptive phrase — existing tools cannot handle it.

[pause]

Here is how QCViz solves this.

MolChat is a structure grounding backend we built from scratch. It is not a simple wrapper around an external API.

It uses a multi-step fallback chain to get 3D structures automatically. [pause]

Step one: scan databases like PubChem. If 3D coordinates exist, use them directly. Step two: if the database has no 3D data, generate a 3D structure using RDKit (R-D-kit) conformer generation. Step three: optimize the generated structure with xTB (x-T-B) to confirm it is chemically reasonable.

If the input is ambiguous, the system presents candidates and waits for user confirmation. [pause] Until the user confirms, the structure is not locked. No calculation runs on an unverified structure. This is a safety mechanism.

Here is why this matters — and frankly, this alone has novelty. [pause]

The user can say "acetone" (AS-uh-tone), "아세톤," or "that ketone." It all works. Database fragmentation, 3D conversion, and structure optimization are handled automatically in the background.

The structure preparation step — which was tedious even for computational chemists — is now automated.

Takeaway: MolChat turns any molecular name or description into a verified, compute-ready 3D structure, automatically.

[Slide 7: LangChain Orchestration — Translating Questions into Compute Requests]

Now, let's move to the orchestration layer. [pause]

Here is the existing bottleneck. When an experimentalist says, "Show me the HOMO of this molecule," a computational chemist had to translate that mentally.

Identify the molecule. Decide the task type. Choose a basis set. Write the input file in the correct syntax. If information is missing, ask again. [pause]

This translation, judgment, and setup process itself consumed one person's time as manual labor. And if the question changed, or a follow-up came in, the process restarted.

[pause]

Here is how QCViz solves this.

A LangChain-based multi-step pipeline automates the translation process that a computational chemist used to do manually.

Step one — molecule identification. "아세톤," "that ketone," and "acetone" all resolve to the same molecule. Step two — task classification. The system routes the request to HOMO, ESP, or optimization. Step three — setting selection. The functional, basis set, and grid are chosen automatically based on the question type and the molecule. Step four — clarification check. If information is missing, the system asks the user. If everything is clear, it proceeds. Step five — session persistence. If the user says "now show the ESP for the same structure," the system carries the previous context forward.

[pause]

One important design principle about the LLM's role. [pause]

The LLM is a translator, not a calculator. An LLM can generate plausible-looking energy values without running any computation. In a research tool, that is the most dangerous failure mode.

In this system, the LLM only handles translation. Every number must come from PySCF. We enforced this by design.

Takeaway: LangChain automates the expert's translation work. The LLM translates language. PySCF produces numbers. These roles never mix.

[Slide 8: Automatic Setup Selection — So You Don't Need to Know B3LYP/6-31G(d)]

Now let's look at automatic setup selection. [pause]

Here is the existing bottleneck. To run a calculation, you must choose a functional, a basis set, a grid, convergence criteria (con-VER-jens cry-TEER-ee-uh), and dispersion correction.

"Should I use B3LYP (bee-three-LIP) or M06-2X (M-oh-six-two-X) for this molecule?" "Is 6-31G(d) (six-thirty-one-G-d) enough, or do I need cc-pVTZ (see-see-p-V-T-Z)?" "Does this molecule have heavy atoms that need an ECP (E-C-P)?"

These decisions require experience from running many calculations. Textbooks alone are not enough. [pause] Experimentalists get completely stuck here. And even computational chemists find it draining to repeat "use these settings" for every new request.

[pause]

Here is how QCViz solves this.

The system reads the molecular properties — element types, size, charge, and spin — together with the question type — orbital, ESP, or optimization. Then it selects a reasonable default automatically.

For a typical organic molecule with an orbital question, it selects B3LYP/6-31G(d). If the molecule contains a transition metal, it decides whether to apply an ECP. For large molecules, it adjusts the basis set level and grid density. If non-covalent interactions matter, it decides whether to include D3 or D4 dispersion correction.

This uses a combination of rule-based logic and LLM-assisted judgment.

[pause]

Experts can override any setting. But the goal is that the default itself is a reasonable first choice — one that a computational chemist would find acceptable.

Takeaway: The user does not need to know B3LYP/6-31G(d). The system picks a reasonable starting point. Experts can still adjust.

[Slide 9: PySCF — The Engine That Actually Runs the Numbers]

Now let's talk about the compute engine itself. [pause]

Once the structure is confirmed and settings are selected, PySCF runs the actual electronic structure calculation. It supports DFT, Hartree–Fock (HAR-tree FOCK), and some post-HF methods.

Every energy value, every orbital, and every ESP map the user sees comes from PySCF. Not from an LLM-generated sentence.

[pause]

Why PySCF? Three reasons.

First, it is open-source. We can embed it in a web platform with no license restrictions. Second, it is Python-native, so pipeline integration is natural. Third, it has an active development community and a growing extension ecosystem, including GPU4PySCF (G-P-U-four-pie-S-C-F).

[pause]

Now, on cross-validation. [pause]

The GPU4PySCF benchmark shows single-digit microhartree (MY-kro-HAR-tree) agreement with Q-Chem 6.1 across multiple functionals and basis sets.

Li et al. in the Journal of Physical Chemistry A, 2025, report microhartree-level agreement with ORCA (OR-kuh) and LSDalton (L-S-DAWL-ton) for total energies and atomization energies.

The SciPy (SY-pie) 2025 comparison with SIESTA (see-ES-tah) shows chemical accuracy maintained with performance gains.

[pause]

I should be honest here. A single dedicated paper comparing PySCF versus ORCA head-to-head is still limited. We acknowledge this.

But the overall picture is: sufficient cross-validation evidence exists, and we can explain under which conditions differences arise.

Takeaway: PySCF is the compute engine. It is open-source, Python-native, and cross-validated against major codes at microhartree accuracy.

[Slide 10: Browser-Native Visualization and Interpretation Metrics]

Now let's move to the last block of the pipeline — visualization. [pause]

Here is the existing bottleneck. Even after the calculation finishes, the work is not done. You must take the output files — .fchk, .molden (MOLE-den), .cube — and open a separate program.

For orbitals, you open GaussView (GOSS-view) or Avogadro (ah-voh-GAH-dro). For ESP, you open VMD (V-M-D). For structure comparison, you open yet another viewer.

[pause]

Most of these programs require a Linux environment. Installation itself is an advanced programming task — dependencies, compilation, environment variables, licenses. Each viewer has its own file format, its own settings, and its own OS compatibility.

Switching between two or three programs to view one set of results is routine. [pause]

For experimentalists, this is a double barrier: they cannot run the calculation, and they cannot view the result.

[pause]

Here is how QCViz solves this.

The moment a calculation finishes, results render directly inside the browser. [pause]

No installation. No Linux. No file conversion. No viewer switching.

3D orbital visualization — rotation, zoom, and iso-value adjustment — all in the browser. ESP maps — charge distribution on the molecular surface — in the same window. Energy values, convergence data, and comparison metrics appear next to the structure.

LLM-assisted interpretation provides text like: "This HOMO is mainly distributed around the carbonyl oxygen."

Calculation, visualization, and interpretation — all in one screen. Just open a browser.

Takeaway: QCViz eliminates the viewer chain. Calculation results appear instantly in the browser, ready for interpretation.

[Slide 11: Live Demo 1 — "Show Me the HOMO of Benzene"]

Now I will show a live demo. [pause] One simple question. The full pipeline. Real results.

The user types: "Show me the HOMO orbital of benzene." [pause]

Here is what happens inside.

LangChain parses: molecule equals benzene, task equals HOMO visualization. MolChat retrieves the benzene 3D structure and locks it. Automatic setup selects B3LYP/6-31G(d). PySCF runs and generates orbital data. The browser renders the 3D HOMO orbital, displays the energy value, and shows a summary text.

[pause]

The user asked one question. The user touched almost no intermediate step. The user received an interpretable screen.

With the old workflow, this would require: structure preparation, input file writing, CLI execution, output parsing, opening Avogadro, and making the figure. At least six steps.

Takeaway: One question in, one interpretable result out. That is the core experience.

[Slide 12: Live Demo 2 — Three Calculations, One Conversation]

This second demo shows that the platform's value is not a single function. It is continuity. [pause]

Here is the flow.

Turn one: "Show me the HOMO of benzene." The structure locks. HOMO calculation runs. Visualization appears.

Turn two: "Show me the ESP for the same structure." No new molecule search. The structure is reused. Only the task changes to ESP. Calculation runs. Visualization appears.

Turn three: "Run an optimization too." Same session. Same structure. Optimization runs. Results appear.

[pause]

With the old workflow, each of these would be almost independent tasks. Prepare the structure again. Write the input again. Open the viewer again.

In QCViz, the research context continues without interruption inside one conversation.

In real research, questions do not end at one. [pause] After you see the HOMO, you want the ESP. Then you want to swap a substituent. Then you want to optimize. This natural flow is what the system supports.

Takeaway: Real research asks chains of questions. QCViz keeps the chain going in one session.

[Slide 13: End-to-End Success Rate Across Scenarios]

Now, results. Not "it works well." Instead, "under these conditions, here are the numbers." [pause]

We tested end-to-end success rates across multiple scenarios.

Single-molecule HOMO queries. Single-molecule ESP queries. Geometry optimization queries. Follow-up queries with structure reuse. Ambiguous inputs — descriptive phrases, abbreviations, Korean text.

And edge cases — nonexistent molecule names, incomplete questions, unclear intent.

[pause]

For each scenario, we report the number of attempts, the number of successes, and a classification of failure causes.

This table shows transparently where the system is strong and where it is still weak.

Takeaway: We report success rates by scenario with failure analysis. No claims without numbers.

[Slide 14: What Actually Changes When This Works]

Let me now talk about impact — what changes when this system works. [pause]

For experimentalists: [pause] They are no longer people who request a calculation and wait. They become people who can start the first exploration themselves. That word — "first" — is key. They apply concepts like HOMO, ESP, and optimization to their own molecules directly. They build intuition by doing. Computation becomes a daily tool, not an outsourced task.

For computational experts: [pause] They are freed from repetitive setup and simple request handling. They can focus on what truly needs their expertise — validation criteria, result interpretation, and method selection.

For the entire lab: [pause] The turnaround time from question to first result gets shorter. Computation usage frequency goes up. Friction between experiment and computation goes down. Computation can become a shared lab tool, not one person's dedicated job.

Takeaway: QCViz does not replace the expert. It removes the friction so that everyone can start, and experts can focus.

[Slide 15: Where QCViz Sits in the Literature — and Where It Could Go]

Let me now position this work in the literature. [pause]

This project does not propose a new electronic structure theory. It redesigns the access path for experimentalists to reach computation. It sits at the intersection of computational chemistry, scientific workflow automation, and human-AI interaction.

[pause]

Compared to existing systems:

El Agente and ChemGraph are agents — CLI-based, designed for experts. QCViz is a platform — browser-based, designed for experimentalists.

WebMO is web-based, but it lacks a natural language interface, automatic setup selection, and integrated visualization.

No existing system combines structure grounding, automatic settings, real DFT, browser visualization, and interpretation metrics in one seamless screen. [pause]

For target journals: [pause]

The Journal of Chemical Information and Modeling — JCIM — is the traditional home for platform and workflow papers. It is the most realistic and fitting target.

The Nucleic Acids Research Web Server Issue offers maximum impact if QCViz is deployed as a public web server. This is the SWISS-MODEL model.

Digital Discovery from RSC is a newer journal focused on LLM plus scientific automation.

Takeaway: QCViz fills a specific gap in the literature — a seamless, browser-based, experimentalist-facing platform for electronic structure calculation.

[Slide 16: Limitations Now, Roadmap Next]

Let me be honest about current limitations. [pause]

CPU-based PySCF is not always the fastest code. Older benchmarks report cases where Gaussian and ORCA are faster.

GPU4PySCF integration would bring major speedup, but it is not fully connected to the platform yet.

A dedicated PySCF-versus-ORCA comparison paper is still limited.

The clarification flow stability and result explanation quality are still improving.

Automatic setup selection is not optimal for every system. A universally correct default does not exist.

The current focus is isolated molecules. Large systems are not yet supported.

[pause]

Now, the roadmap. [pause]

For workflow depth: transition state and IRC searches, frequency analysis, IR and Raman spectra, thermochemistry (THER-mo-KEM-uh-stree), and NMR prediction.

For system complexity: surface DFT, cluster DFT, and periodic DFT.

For performance: full GPU4PySCF integration, and integration of other DFT codes.

For recent trends: MCP integration for AI agent skill sharing.

[pause]

The current architecture already separates question parsing, structure retrieval, calculation execution, and result delivery. This means we can expand the backend engines and workflows while keeping the frontend user experience the same.

Takeaway: We are transparent about limitations. The modular architecture makes each limitation addressable without redesigning the whole system.

[Slide 17: Three-Sentence Summary]

QCViz is a web platform that lets experimentalists start electronic structure calculations from a browser, in natural language, with no installation.

It is not a chatbot that fabricates numbers. MolChat, LangChain, and PySCF divide roles to perform real computation and visualization.

The value of this project is not automating one calculation. It is making the access path to computation shorter and more frequently usable across the lab.

[Slide 18: Thank You — Questions & Discussion]

Thank you for your attention. I am happy to take questions.

[Pronunciation Notes]

Term	Pronunciation
PySCF	pie-S-C-F
HOMO	HO-mo
LUMO	LOO-mo
ESP	E-S-P
DFT	D-F-T
Hartree–Fock	HAR-tree FOCK
B3LYP	bee-three-LIP
M06-2X	M-oh-six-two-X
6-31G(d)	six-thirty-one-G-d
cc-pVTZ	see-see-p-V-T-Z
ECP	E-C-P
LangChain	LANG-chain
MolChat	MOL-chat
RDKit	R-D-kit
xTB	x-T-B
ORCA	OR-kuh
SIESTA	see-ES-tah
microhartree	MY-kro-HAR-tree
GPU4PySCF	G-P-U-four-pie-S-C-F
convergence criteria	con-VER-jens cry-TEER-ee-uh
Avogadro	ah-voh-GAH-dro
GaussView	GOSS-view
thermochemistry	THER-mo-KEM-uh-stree
iso-value	EYE-so-VAL-yoo
SciPy	SY-pie
LSDalton	L-S-DAWL-ton
SMILES	smylz
.molden	MOLE-den

[Delivery Tips — Section by Section]

Slides 1–4 (Introduction): These slides sell the *problem*. Speak slowly and let each pain point land. When listing the seven steps on Slide 1, count them on your fingers or use a pointer — the audience should feel the weight of the list. On Slide 4, when presenting the table of published platforms, do not rush through every entry. Pick two or three to emphasize verbally and let the audience read the rest.

Slides 5–10 (Methods): Pace yourself — this is the longest section. At the start of each slide, glance at the audience and deliver the first sentence directly to them before turning to the screen. On Slide 7, when you say "The LLM is a translator, not a calculator," pause and look at the audience. This is a key trust-building line. On Slide 9, when acknowledging limited head-to-head comparison papers, keep your tone matter-of-fact — honesty delivered calmly builds credibility.

Slides 11–13 (Results / Demo): If doing a live demo, rehearse the exact inputs beforehand so there are no typos on stage. If showing recorded demos, narrate each step as it plays — do not let the video speak for itself. On Slide 13, point directly at the table and walk through one row fully, then let the audience scan the rest.

Slides 14–18 (Conclusion & Outlook): On Slide 14, slow down at "first exploration" — emphasize the word "first." On Slide 16, limitations, keep your tone confident. Listing limitations is a strength, not a weakness. On Slide 17, the three-sentence summary, consider memorizing these three sentences and delivering them without looking at the screen. This is your closing impression. Make eye contact.